Scoring & Stats Settings

How player rankings are calculated in Sideline, and what every admin setting does

Reference guide · generated 21 July 2026

> Every player carries a **rating from 0–100**. It blends two independent signals — what teammates think of them (**peer votes**) and what they actually did on the pitch (**approved stats**) — and then dampens the result based on how much data we have, so a player with two votes can never leapfrog someone with two seasons of evidence.

Contents

1. The 0–10 rating scale
2. The big picture: how a final score is built
3. Confidence tiers
4. Peer score, step by step
5. Stats score, step by step
6. Worked end-to-end example
7. Admin settings reference
8. Advanced fairness settings
9. Per-match: “Include stats in scoring”
10. Quick glossary

1. The 0–10 rating scale

When voting after a match, players rate each teammate from **0 to 10**. Each number has a shared descriptor so everyone interprets the scale the same way. **5 is average** — an ordinary, solid performance, not a bad one.

Rating	Descriptor	Tier
0	Didn't turn up	Low
1	Very poor	Low
2	Poor	Low
3	Below average	Low
4	Fair	Mid
5	Average	Mid
6	Decent	High
7	Good	High
8	Very good	High
9	Excellent	Elite
10	World class	Elite

These 0–10 votes are later mapped linearly onto the 0–100 scale (a raw vote average of 7.0 → 70).

2. The big picture

A player's final score is a weighted blend of two adjusted sub-scores:

```
final = peerAdjusted × peerWeight  +  statsAdjusted × statsWeight
```

- **Peer weight** and **stats weight** always add up to 100%. The default is **Peer 60% / Stats 40%**.
- If a player has **no approved stats**, stats weight becomes 0 and the final score is 100% peer-based.
- Both sub-scores are *damped toward a neutral score (default 50)* when we have little data, so new or thinly-rated players sit near the middle instead of at the extremes.

Why "damping toward neutral"?

A single 9/10 vote shouldn't make someone the best player in the club. Instead of trusting a raw average blindly, we scale it by a **confidence factor** between 0 and 1. Low confidence pulls the score back toward 50; full confidence lets the raw score through unchanged.

```
adjusted = neutral + confidence × (raw − neutral)
```

3. Confidence tiers

Every player is labelled with a tier that reflects how trustworthy their score is:

Tier	Meaning	Default rule
unrated	Not enough matches played yet.	Fewer than **1** match
provisional	Playing, but few peer votes so the score may still swing.	Fewer than **8** valid peer votes
rated	Enough data for a stable, trustworthy score.	**8+** valid peer votes

Tiers are informational — they don't change the maths, but they tell you how much to trust a number.

4. Peer score, step by step

The peer score turns raw teammate votes into a damped 0–100 value. Five stages:

Step 1 — Collect valid votes per match

Only votes flagged as in-scope / included in calculation are counted. Votes an admin has removed or marked out-of-scope are ignored.

Step 2 — Remove outliers (per match)

Within a single match, we drop rogue votes using the interquartile-range (IQR) rule — the same technique used to ignore one friend's spite-vote or one buddy's inflated 10.

- Fewer than 4 votes: keep them all (too few to judge outliers).
- Otherwise compute Q1 and Q3, and keep only votes within `[Q1 − 1.5·IQR, Q3 + 1.5·IQR]`.
- If filtering leaves fewer than 3 votes, fall back to the median of the original votes.

Step 3 — Aggregate each match into one number

The surviving votes for a match are collapsed to a single value. By default this is the **mean**; admins can switch to **median** or **trimmed mean** (see Advanced fairness).

Step 4 — Combine matches (recency-weighted)

A player's per-match values are averaged with the **three most recent matches counting double**, so current form matters more than old results. The result (0–10) is multiplied by 10 to land on 0–100.

```
peerRaw(0–100) = weightedAverage(perMatchValues) × 10
```

Step 5 — Damp by confidence

Two confidence factors are multiplied together, then applied:

- **Vote confidence** — `min(1, validVotes / peerFullConfidenceVotes)` (default full at 12 votes).
- **Match confidence** — `min(1, matchesPlayed / matchCountFullConfidence)` (default full at 3 matches).

```
peerAdjusted = 50 + (voteConf × matchConf) × (peerRaw − 50)
```

> **Example — a promising newcomer**
>
> Priya has played **2 matches** and received **6 valid votes** averaging **8.0/10**.
>
> - peerRaw = 8.0 × 10 = **80**
> - voteConf = min(1, 6 / 12) = **0.5**
> - matchConf = min(1, 2 / 3) = **0.667**
> - combined confidence = 0.5 × 0.667 = **0.333**
> - peerAdjusted = 50 + 0.333 × (80 − 50) = 50 + 10 = **60.0**
>
> Even with glowing 8s, her thin sample keeps her at a modest 60 — she'll climb as more data arrives.

5. Stats score, step by step

The stats score rewards on-pitch output — goals, assists and penalties — weighted by playing position. It only uses **approved** stats from matches where stats scoring is enabled.

Step 1 — Score each match with position multipliers

Each stat is multiplied by a position-specific factor (inspired by Fantasy Premier League). A goal from a goalkeeper is worth more than a goal from a forward, because it's rarer and more valuable.

Position	Goal	Assist	Penalty scored	Penalty conceded
GK (goalkeeper)	2.0	1.0	1.5	−1.0
DEF (defender)	1.5	1.0	1.3	−1.3
MID (midfielder)	1.25	1.0	1.2	−1.0
FWD (forward)	1.0	1.2	1.0	−0.8

```
matchStat = goals·Mgoal + assists·Massist + pensScored·Mpen+ + pensConceded·Mpen−
```

Step 2 — Average matches (recency-weighted)

As with peer votes, the three most recent matches count double.

Step 3 — Normalise across the squad

Raw stat totals are stretched onto a 0–100 scale relative to the rest of the club: the lowest becomes 0, the highest becomes 100, everyone else sits in between. (If everyone is tied, an absolute anchor is used: 0 → 50, 5 → 75, 10 → 100.)

Step 4 — Damp by confidence

Same idea as peer: `min(1, statMatches / statFullConfidenceMatches)` (default full at 4 matches), multiplied by match confidence, then applied around neutral.

Example — a clinical midfielder

Sam (MID) across 4 stat-tracked matches averaged **2 goals + 1 assist** per game.

- matchStat = 2 × 1.25 + 1 × 1.0 = **3.5** per match
- Suppose that normalises to **85** against the squad's spread.
- statConf = min(1, 4 / 4) = **1.0**, matchConf = min(1, 4 / 3) = **1.0**
- statsAdjusted = 50 + 1.0 × (85 − 50) = **85.0**

6. Worked end-to-end example

Putting peer + stats together (default 60/40 blend)

Take a `rated` player with:

- peerAdjusted = **72**
- statsAdjusted = **85**

```
final = 72 × 0.60 + 85 × 0.40
      = 43.2 + 34.0
      = 77.2
```

If that same player had **no approved stats**, stats weight would drop to 0 and their final score would simply be their peer score: **72.0**.

7. Admin settings reference

These live on the **Scoring** admin page. Changing them recomputes everyone's score.

Peer / stats blend

Setting	Default	What it does
Stats weight (%)	40	Share of the final score driven by stats. Peer weight is automatically 100 − this. Slide toward stats for a more "results-based" club, toward peer for a more "eye-test" club.

Confidence thresholds

Setting	Default	What it does
Unranked min matches	1	Matches a player must play before leaving the `unrated` pool.
Provisional min valid peer votes	8	Valid votes needed before a rating stops being `provisional` and becomes `rated`.
Peer full-confidence votes	12	Vote count at which the peer score is fully trusted (no more damping from the vote side). Lower = quicker to trust votes.
Match-count full confidence	3	Matches at which overall data is fully trusted. Affects both peer and stats damping.
Neutral score	50	The value low-confidence scores are pulled toward. This is also where brand-new players effectively start.

Example — making votes count faster

Drop *Peer full-confidence votes* from 12 to 6. Priya's earlier example (6 votes, raw 80, 2 matches) changes: voteConf becomes `min(1, 6/6) = 1.0`, so combined confidence = 1.0 × 0.667 = 0.667, and peerAdjusted jumps from 60.0 to **70.0**. Great for a small, close-knit group; riskier if a few voters can dominate.

Awards thresholds

Setting	Default	What it does
Per-match minimum valid votes	—	Minimum votes a player needs in a single match to qualify for that match's awards (e.g. Man of the Match).
Overall minimum valid votes	—	Minimum total votes to qualify for season / overall awards.

8. Advanced fairness settings

These are optional refinements. **All default to OFF**, which reproduces the classic scoring behaviour exactly. Turn them on to make scores fairer or more robust — but note that enabling any of them changes the numbers by design.

Normalize voter harshness

Re-centres each voter's ballot so their personal average matches the match average. This cancels out habitually harsh or lenient voters while preserving the match's overall mean.

> **Example**
>
> In a match averaging 6.5, a chronically stingy voter gives everyone ~1 point low (their own average is 5.5). Normalization adds `6.5 − 5.5 = +1.0` to each of their votes, so their opinions still count but their harshness no longer drags players down.

Weight matches by number of voters

A match rated by 14 people is stronger evidence than one rated by 3. When on, each match's influence is scaled by how many people voted (on top of the recency double-weight).

Per-match vote aggregate

Option	Behaviour
Mean (default)	Simple average of surviving votes.
Median	Middle value — maximally robust to a couple of extreme votes.
Trimmed mean	Drops the top and bottom 10%, then averages the rest — a balance of the two.

> **Example**
>
> Votes `[4, 7, 7, 8, 10]` : mean = 7.2, median = 7, trimmed mean (drop one each end) = 7.33. Median resists that lone 10 the most.

Bayesian shrinkage

A smoother alternative to the hard vote-threshold ramp. It blends a player's raw score with a neutral prior worth *k* "pseudo-votes", so scores glide toward neutral when data is thin instead of stepping.

```
shrunk = (validVotes × peerRaw + k × 50) / (validVotes + k)
```

> **Example (k = 6)**
>
> 4 votes averaging raw 90: `shrunk = (4×90 + 6×50) / (4+6) = (360+300)/10 = 66` . The score is pulled from 90 toward neutral because only 4 votes back it. *Prior strength (k)* controls how hard: higher k = more shrinkage.

Recency decay for inactivity

Pulls long-absent players back toward neutral so the table reflects *current* form. Confidence decays with a configurable half-life.

```
decay = 0.5 ^ (daysSinceLastMatch / halfLifeDays)
```

> **Example (half-life = 45 days)**
>
> A player who hasn't played for 90 days: `decay = 0.5^(90/45) = 0.5² = 0.25`. Their confidence — and therefore how far their score sits from neutral — is quartered until they play again.

9. Per-match: "Include stats in scoring"

Each match has an **Include stats in scoring** checkbox in its settings. When unticked, that match's goals/assists are recorded and displayed but **excluded from the stats score**. Use it for friendlies, lopsided teams, or early sessions where the numbers would distort rankings. Peer votes from the match are unaffected.

10. Quick glossary

Term	Meaning
Valid vote	A peer vote that is in-scope and included in calculation (not removed by an admin).
Neutral score	The midpoint (default 50) that low-confidence scores are pulled toward.
Confidence factor	A 0–1 multiplier that decides how much of a raw score is trusted.
Recency weighting	The three most recent matches count double when averaging.
Outlier filtering	IQR-based removal of rogue votes within a single match.
Normalization	Stretching raw stat totals onto a 0–100 scale relative to the squad.

Sideline scoring reference. Defaults shown reflect the shipped configuration; your club's admin may have changed them on the Scoring page. All "Advanced fairness" options are off unless explicitly enabled.